

notebook

September 22, 2025

1 GLM-HMM fit to data

Notebook to fit the GLM-HMM from Ashwood et al. (2020) ([Mice alternate between discrete strategies during perceptual decision-making](#)).

Based on the example notebook [2b Input Driven Observations \(GLM-HMM\)](#).

The logic of the original notebook is to simulate synthetic data with a generative GLM-HMM, fit a GLM-HMM to the synthetic data, and check that we can recover the generative parameters. Following that tutorial, we will use real data from my 2AFC task instead, fit a GLM-HMM to it and recover the parameters. Easy peasy, lemon squeezy!

1.1 1. Setup

The line `import ssm` imports the package for use. Here, we have also imported a few other packages for plotting.

1.2 2. Input Driven Observations

We create a HMM with input-driven observations and ‘standard’ (stationary) transitions with the following line:

```
ssm.HMM(num_states, obs_dim, input_dim, observations="input_driven_obs", observation_k
```

As in Ashwood et al. (2020), we are going to model an animal’s binary choice data during a decision-making task, so we will set `num_categories=2` because the animal only has two options available to it. We will also set `obs_dim = 1` because the dimensionality of the observation data is 1 (if we were also modeling, for example, the binned reaction time of the animal, we could set `obs_dim = 2`). For the sake of simplicity, we will assume that an animal’s choice in a particular state is only affected by the external stimulus associated with that particular trial, and its innate choice bias. Thus, we will set `input_dim = 2` and we will simulate input data that resembles sequences of stimuli in what follows. In Ashwood et al. (2020), they found that many mice used 3 decision-making states when performing 2AFC tasks. We will, thus, set `num_states = 3`.

1.2.1 Initialize GLM-HMM (2a)

1.3 3. Fit GLM-HMM and perform recovery analysis

Instead of generating data, we will use real data from my 2AFC task.

The **action trace** C_t is a measure of how past choices influence current behavior, with more recent choices having a greater impact. It is calculated as an exponentially weighted sum of past choices,

where the weights decay over time according to a time constant τ . The formula for the action trace at trial t is given by:

$$C_t = \sum_{i=1}^k e^{-k/\tau} \cdot r_{t-i}$$

To prepare the data for the GLM-HMM, we need to organize both the inputs (external covariates) and the choices (observations) **as lists of arrays, one per session**.

In this example, `df` contains the data from one or multiple sessions concatenated.

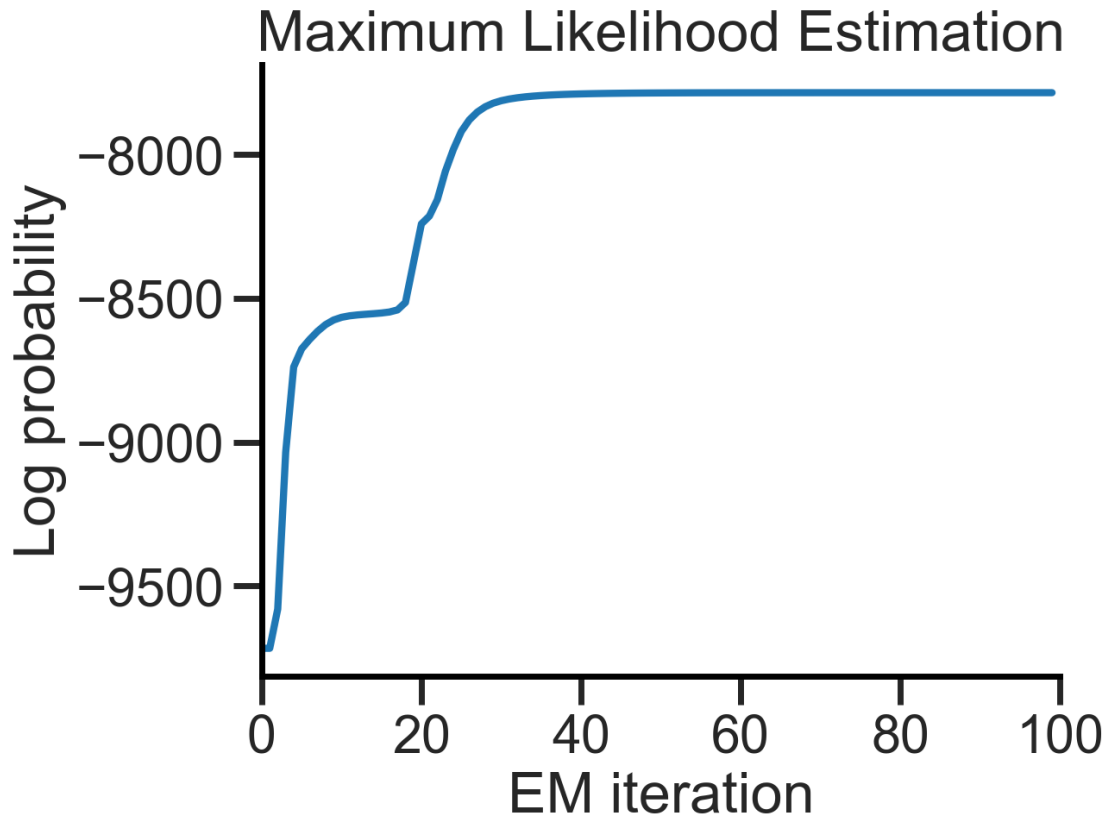
- **Inputs:** For each session, we create a 2D array of shape (N_i, D) , where N_i is the number of trials in that session and $D = 2$ is the number of input dimensions. Here, the two columns are:
 1. `df.ILD.values` — the stimulus presented on each trial
 2. A constant column of 1s — to capture the animal's baseline bias
- **Choices:** For each session, we create a 2D array of shape $(N_i, 1)$, where N_i is the number of trials in that session. This array contains the animal's choices (0 or 1) for each trial.

Check parsed data has correct format and shape

3a. Maximum Likelihood Estimation

Now we fit the GLM-HMM and check that we can recover the parameters in the data:

```
0%|          | 0/200 [00:00<?, ?it/s]
```



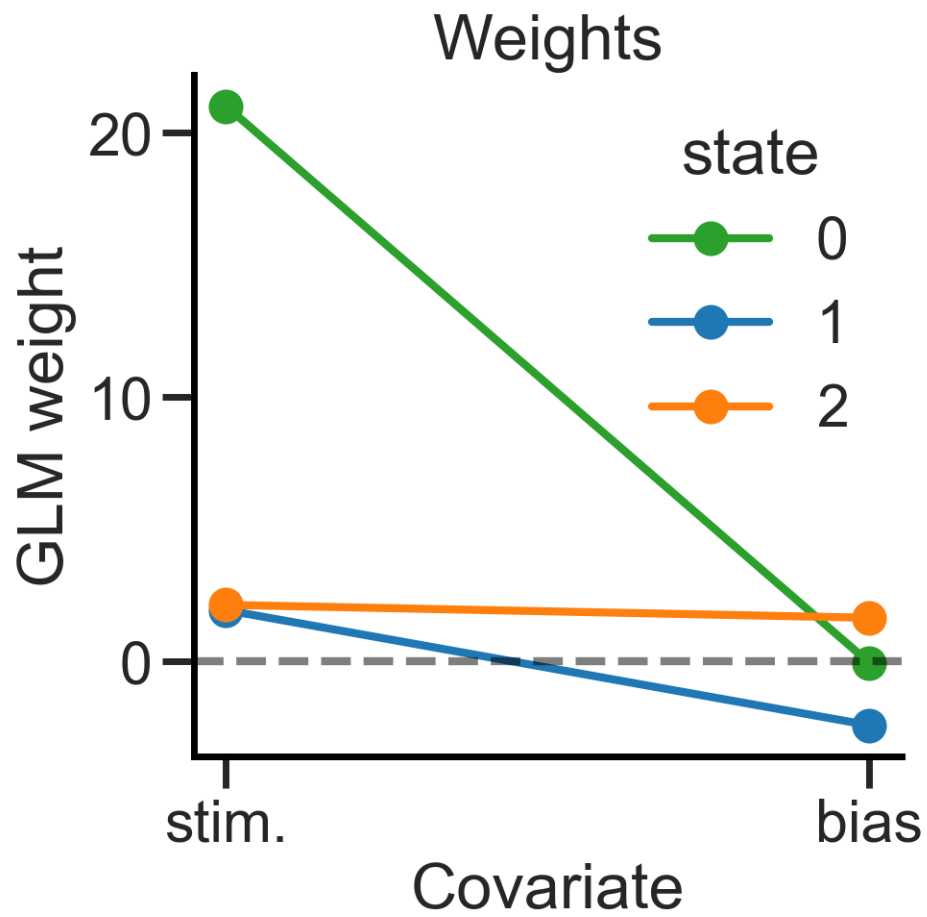
1.3.1 3b. Retrieved parameters

State 1: engaged (stim. weight: 21.01, bias weight: -0.05)

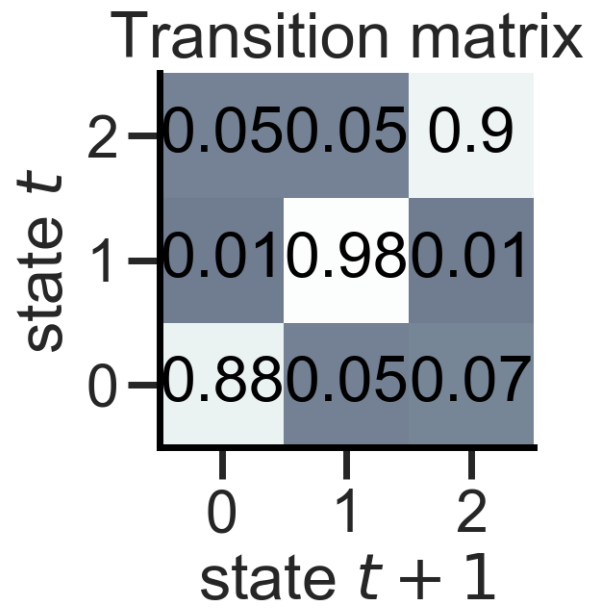
State 2: biased left (stim. weight: 1.94, bias weight: -2.41)

State 0: biased right (stim. weight: 2.14, bias weight: 1.66)

Text(0.5, 1.0, 'Weights')

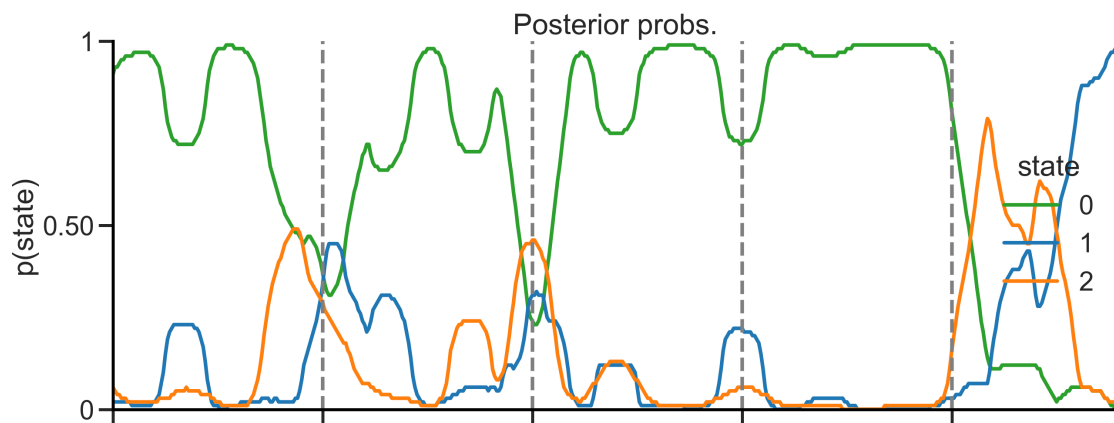


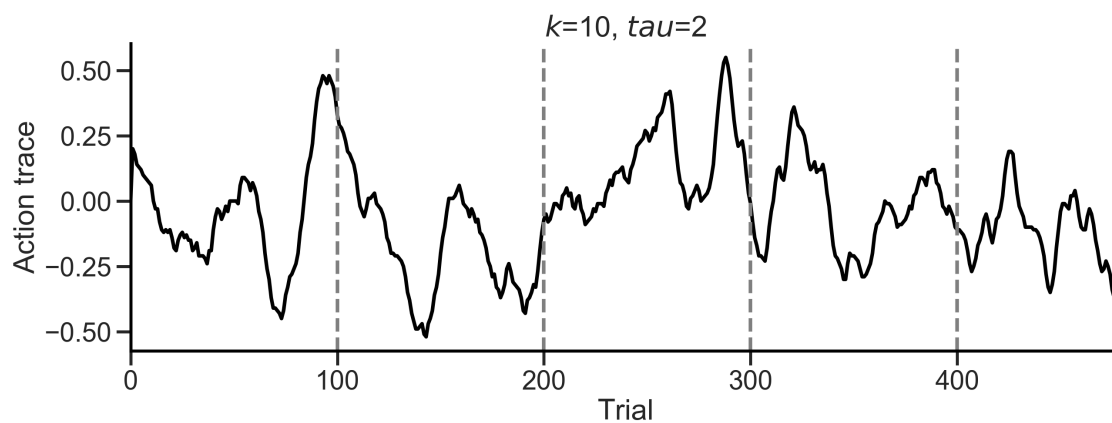
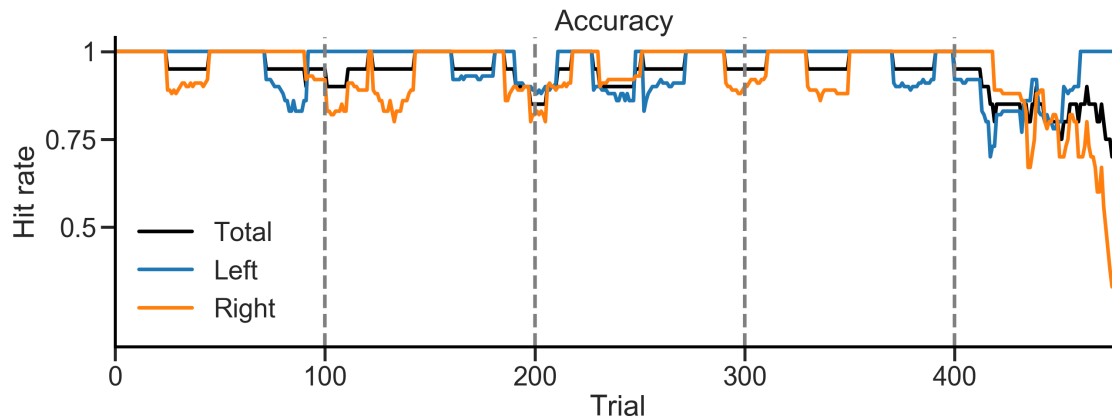
Text(0.5, 1.0, 'Transition matrix')



1.3.2 3c. Posterior State Probabilities

Let's now plot $p(z_t = k | \mathbf{y}, \{u_t\}_{t=1}^T)$, the posterior state probabilities, which give the probability of the animal being in state k at trial t .



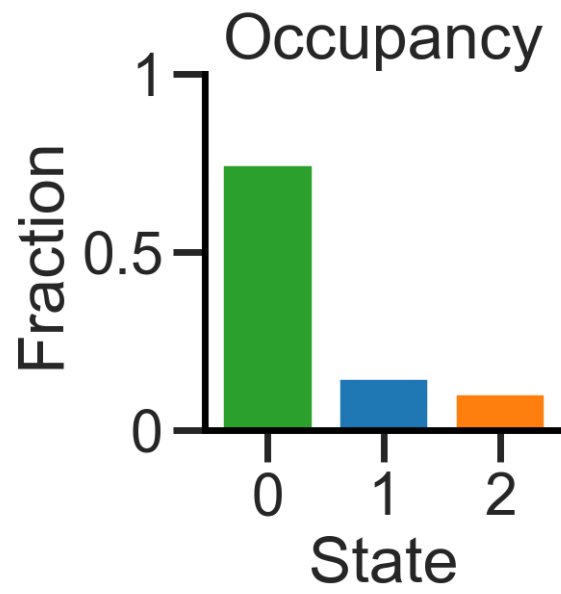


With these posterior state probabilities, we can assign trials to states and then plot the fractional occupancy of each state:

State 1 occupancy: 0.75

State 2 occupancy: 0.15

State 3 occupancy: 0.10



2 Interpret states and remap

<matplotlib.legend.Legend at 0x2082a552508>

